```
const formatted = await prompt.format({
  input: "What is the capital of Canada?",
});
```

The format method generates a complete prompt by inserting user input into the template.

If you run this example from the command line, you will see the formatted output that can be used as a prompt. Run it from the root folder of the project source code using the following command:

```
$ node few-shot-chain.js
```

This would print the prompt template in the console, which you can review and adjust if needed.

Now that we have a robust method for constructing various structured prompts and a way to compose chains, we can explore how to integrate these functionalities into our web application using the Vercel AI SDK.

6.1.2 Integration with the Vercel AI SDK

Integrating LangChain's features with the Vercel AI SDK allows developers to utilize the best of both worlds when building generative AI web apps. Recall that, in the current state, the Vercel AI SDK is best used for handling interactions with structured data, streaming UI elements, and utilizing various tools.

However, its primary limitations lie in its focus on seamless integration with Next.js server components and the abstractions it provides for efficient UI updates. While these features are beneficial, tasks such as document ingestion and retrieval, prompt management, and chaining calls are left to the developer to implement independently. As an improvement to our existing knowledge, we want to utilize better abstractions that deal with more complex scenarios, such as handling large datasets by indexing documents or consolidating complex retrievals of relevant search queries.

This is where LangChain becomes an invaluable addition. We next explore how to integrate LangChain into our project.

INTEGRATING PROMPT TEMPLATES

Integrating a prompt template exported by the LangChain library into the existing project is a straightforward process. We can take the previously created few-shot prompt template and import it into the actions.jsx file, which handles the server-side logic. By formatting the prompt with user input—representing the user's query—we can seamlessly incorporate it into the `streamUI` history messages. The following listing shows the changes to the actions.jsx file.

Listing 6.3 Integrating prompt template into the conversation handler

```
import { prompt } from '../../lib/fewShotPrompt';

export async function continueConversation(input, provider, model) {
```

Imports the few-shot prompt template from the specified module